Capstone Project · Presentation Script (Bilingual)

Project: Full-Stack Portfolio & Blog Application

Course: Web Data Management and Application

Author: Yu Shanna

Length: 12–15 minutes including a live demo (3 minutes)

> *How to use this script: read Part 1 if you present in Chinese, Part 2 if you present in English.*
>
> *The numbers match, so you can switch languages mid-way if the examiner asks a question in the other language.*
>
> *Lines in [brackets] are stage directions — do not read them out.*

时间分配 / Timing at a glance

#	内容 / Section	时长	Slide
1	开场 / Opening	0:30	1
2	项目是什么 / What and why	1:00	2
3	技术栈 / Tech stack	1:00	3
4	架构与目录 / Architecture	1:30	4
5	数据模型 / Data models	2:00	5–6
6	认证与授权 / Auth & authorization	2:00	7–8
7	API 端点 / API endpoints	1:30	9
8	错误处理与安全 / Errors & security	1:00	10
9	前端架构 / Front-end architecture	1:30	11
10	设计与交互 / Design & UX	1:00	12
11	现场演示 / Live demo	3:00	—
12	难点与收获 / Challenges	1:00	13
13	部署与提交 / Deploy & submit	0:45	14

14	结尾 / Closing	0:30	15

PART 1 · 中文讲解稿

1. 开场（30 秒）

各位老师好，我是 Yu Shanna。我今天要讲的是本课程的期末项目——一个完整的全栈个人作品集与博客网站。

它由两部分组成：一个用 Node.js 和 Express 写的 REST API，和一个用 React 写的单页应用。网站已经部署上线，代码也已经提交。

我今天的安排是：先用六分钟左右讲清楚这个系统是怎么设计的，然后现场演示一遍完整流程，最后讲我在做的过程中遇到的问题和收获。

2. 项目是什么（1 分钟）

[切到第 2 页：两张截图]

先看一下它长什么样。左边是访客看到的页面：首页、项目作品集、博客列表和文章详情，还有联系表单。右边是登录之后才看得到的后台：我可以在这里新增、修改、删除项目和文章。

所以它其实是一个「作品集 + 博客 + 内容管理后台」三合一的产品。

我把题目拆成了三个人会问的问题：

第一，访客怎么看到我的内容？——需要公开的读取接口。

第二，我怎么管理这些内容？——需要一个只有我能进的写接口和后台。

第三，别人怎么和我互动？——需要评论区，以及一个能真正把消息存进数据库的联系表单。

这三个问题，正好对应系统的三层需求：**读、写、权限。**

3. 技术栈（1 分钟）

[切到第 3 页：技术栈图]

前端是 React 19，用 Vite 构建，路由用 React Router 7，请求用 Axios，全局状态用 React 自带的 Context API，样式是手写的 CSS 变量体系，没有引入 UI 框架。

后端是 Node.js + Express 5，数据库是 MongoDB Atlas，用 Mongoose 做建模和校验。身份认证用 JWT，密码用 bcrypt 哈希，安全头用 helmet，配置用 dotenv 管理。

这里我想强调一点：**我没有为了用而用。**比如状态管理，我的全局状态只有一个「当前登录用户」，用 Context 就够了，不需要 Redux；再比如样式，我用 CSS 变量做了一个自己的设计系统，既轻量，又能一键切换深浅色主题。

4. 架构与目录（1.5 分钟）

[切到第 4 页：目录树]

后端严格按 MVC 分层：

- `models` 放五个 Mongoose 模型，负责数据结构和校验；
- `controllers` 放业务逻辑，每个资源一个文件；
- `routes` 只做 URL 到 controller 的映射，很薄；
- `middleware` 放两个关键中间件：`protect` 负责验 token，`error` 负责统一错误；
- `utils` 放错误类和 token 生成；
- `config/db.js` 负责连接数据库，`app.js` 组装应用，`server.js` 只做「连上数据库再监听端口」。

前后端是彻底分离的两个工程，通过 `/api` 下的 HTTP 接口通信，前端不知道数据库长什么样，后端也不知道页面长什么样。这样我可以独立开发、独立部署。

5. 数据模型（2 分钟）

[切到第 5 页：ER 关系图]

一共有五个模型，我最想讲的是它们之间的关系。

`User` 是核心，`username` 和 `email` 都要求唯一，`password` 有最小长度限制，并且设了 `select: false`——默认查询不会把密码哈希带出来，只有登录时显式 `select('+password')` 才拿得到。

`Project` 属于某个用户，用 `user` 字段存 ObjectId 并 `ref: 'User'`。

`BlogPost` 的 `author` 同样指向 `User`。

`Comment` 最特别，它同时指向 `User` 和 `BlogPost`——「谁在哪个文章下说了什么」，所以它有两个外键。

`Message` 是联系表单的落库结果，和用户没有关系，因为访客不需要注册就能留言。

这些关系在读取时用 `populate` 还原：项目列表会把作者的用户名带出来，文章列表会带作者名，文章详情会把这篇下面的所有评论一起查出来，每条评论再带上评论者的用户名。

[切到第 6 页：一张 populate 的对比截图]

这是 `populate` 前后的区别。存的是 ID，返回给前端的是可以直接显示的对象，前端不需要再发第二个请求去查用户名。

6. 认证与授权（2 分钟）

[切到第 7 页：注册登录流程图]

认证分三步。

注册时，用户提交用户名、邮箱、密码。模型里的 `pre('save')` 钩子会用 bcrypt 加盐哈希，成本因子 10，数据库里永远不存明文。

登录时，我用邮箱把用户查出来，**显式带上密码字段**，然后用 `bcrypt.compare` 比对。比对成功，用 `jsonwebtoken` 签发一个 token，payload 里只放用户 ID，有效期 30 天，密钥从环境变量读。

之后每个受保护的请求都要在请求头里带 `Authorization: Bearer <token>`。`protect` 中间件会验证签名、把用户查出来、挂到 `req.user` 上；token 缺失或无效，直接返回 401。

[切到第 8 页：403 的截图]

授权是另一件事，很多人会把它和认证混在一起。认证回答「你是谁」，授权回答「你能不能做这件事」。

我的规则是：**文章只有作者本人能改能删。**所以在 `updatePost` 和 `deletePost` 里，我会比较 `post.author` 和 `req.user._id`，不是同一个人就返回 403。

这是我特意用两个账号测过的：bob 去改 alice 的文章，接口返回 403，文章没有任何变化。

7. API 端点（1.5 分钟）

[切到第 9 页：端点表]

整个 API 一共 16 个业务端点，加上健康检查。

- 用户三个：注册、登录、还有 `/api/users/me`——这个是我额外加的，用来在页面刷新后验证本地 token 是否还有效。
- 项目五个：公开的列表和详情，受保护的增、改、删。

- 文章五个：同样公开两个、保护三个，其中改和删带作者校验。
- 评论两个：公开读、登录后写。
- 联系表单一个：公开，任何人可提交。

返回值我保持了 REST 的习惯：创建返回 201 和新建的资源，删除返回一个成功对象，出错统一是 `{ success: false, message }` 加正确的状态码。

8. 错误处理与安全（1 分钟）

[切到第 10 页：错误中间件代码]

所有 controller 出错都只做一件事：`next(err)`。真正的格式化集中在最后一个错误中间件里，它会把几类常见错误翻译成人话：

- Mongoose 的校验失败 → 400，并拼出具体哪个字段不合法；
- 重复的邮箱或用户名（错误码 11000）→ 400，说明是哪个字段重复；
- 非法的 ObjectId、找不到资源 → 404；
- 请求体不是合法 JSON → 400；
- token 无效或过期 → 401；
- 其他未知错误 → 500，并在服务端打印堆栈，但不返回给客户端。

安全方面：helmet 加了安全响应头，CORS 只允许配置的前端域名，所有密钥都在 `.env` 里、不进代码仓库，请求体限制 1MB，另外设置 `trust proxy` 以便在 Render 这类反向代理后面拿到真实客户端信息。

9. 前端架构（1.5 分钟）

[切到第 11 页：路由与组件图]

前端是标准的 React 单页应用，九个页面：首页、项目、博客列表、文章详情、联系、登录、注册、后台，加一个 404。

路由用 React Router 的嵌套路由，公共部分（页头页脚）抽成一个 `Layout`，页面作为 `Outlet` 渲染进去。

全局状态只有登录信息，放在 `AuthContext` 里：登录时把用户和 token 写进 state 和 localStorage，登出时清掉。初始化时会调用一次 `/api/users/me` 验证本地 token，如果已经过期就自动登出——用户不会看到一个「看起来登录了、其实什么都提交不了」的假状态。

所有请求走同一个 Axios 实例：请求拦截器自动加上 `Authorization` 头，响应拦截器遇到 401 就清掉本地登录态并广播一个事件，让界面回到未登录状态。

受保护路由用 `ProtectedRoute` 包起来：没登录就 `<Navigate>` 到登录页，并且记住原来想去哪，登录成功后直接跳回去。

10. 设计与交互（1 分钟）

[切到第 12 页：设计系统截图]

设计上我做了一套 CSS 变量：主色是靛蓝到品红的渐变，配青、琥珀、绿做点缀；背景是一层很淡的彩色光晕加网格。

深浅色主题只改一组变量就能整体切换，选择记在 localStorage 里，默认跟随系统。

交互上做了几个细节：加载时是骨架屏而不是一句「Loading」；空列表给的是带引导按钮的空状态；文章页顶部有阅读进度条；后台的危险操作会二次确认；窄屏下导航变成可横向滑动的单行。

所有动画都遵守 `prefers-reduced-motion`，用户如果关闭了动画，页面会立刻静止，不会影响可读性。

11. 现场演示（3 分钟）

[切到浏览器，按顺序操作，边做边说]

我现在从零走一遍完整流程。

第一步，注册。 打开首页，右上角点 Register，填用户名、邮箱、密码。提交之后会自动登录并跳到后台——因为我注册成功后顺手调了一次登录接口，省掉用户再输一遍。

第二步，发一篇项目。 在后台的 Projects 标签里填标题、描述、图片链接，点 Create。注意页面右上角出现了绿色提示，左边的列表也立刻多了一条。

第三步，回到前台看效果。 点 Projects，刚才那条已经出现在列表里，作者名也带出来了，说明 `populate` 生效了。

第四步，写一篇文章并评论。 发一篇博客，然后打开它的详情页，在评论区留一句话。提交之后评论立刻出现，带我的用户名和时间。

第五步，验证权限。 退出登录，再直接访问 `/admin`——会被重定向到登录页。用另一个账号登录，去改刚才那篇文章，接口会返回 403。

第六步，联系表单。 填一个名字、邮箱和消息，提交，提示成功。这条消息已经作为 Message 存进数据库了。

最后，切一下暗色模式。 点右上角的月亮，整套配色会整体切换，刷新后仍然是暗色。

12. 难点与收获（1 分钟）

[切到第 13 页]

讲三个我印象最深的地方。

第一，**认证和授权的区别**。一开始我以为「登录了就能改」，测试时用第二个账号去改别人的文章才发现这是两个独立的判断。后来我把「谁是你」和「你能不能做」拆成两层，代码反而更清楚了。

第二，**SPA 的深链接**。单页应用里刷新 `/blog/xxx` 会 404，因为服务器上并没有这个文件。解决方式是配置重写规则，Netlify 用 `_redirects`，Vercel 用 `vercel.json`，全部指回 `index.html`。

第三，**部署后的跨域问题**。本地开发时前端在 5173、后端在 5000，CORS 必须放行前端域名；部署到 Vercel 之后这个域名变了，后端的环境变量也要跟着改。这让我理解了「环境变量不是可选项」。

13. 部署与提交（45 秒）

[切到第 14 页]

后端部署在 Render：根目录 `server`，启动命令 `node server.js`，环境变量里配数据库连接串、JWT 密钥和前端域名。

前端部署在 Vercel：构建命令 `npm run build`，产物目录 `dist`，环境变量里配后端的线上地址 `/api`。

数据库是 MongoDB Atlas，在 Network Access 里放行了 Render 的出口地址。

最终提交的压缩包里是 `server`、`client` 两个文件夹加一份 README，README 里写了两个线上地址、全部接口清单和本地运行步骤；按题目要求，`node_modules`、`dist` 和 `.env` 都没有打进去。

14. 结尾（30 秒）

总结一下：这个项目完成了课程要求的全部内容——MVC 结构的 REST API、五个模型、bcrypt 与 JWT、受保护的路由、评论与联系表单、React 的组件化与路由、Context 全局状态、受保护的后台，以及前后端的分别部署。

做完之后我最大的感受是：**前端决定用户看到什么，后端决定用户能做什么，而权限设计决定这个系统值不值得信任。**

我的讲解到这里，谢谢老师，接下来的问题我都可以回答。

PART 2 · English Script

1. Opening (0:30)

Good morning. My name is Yu Shanna, and this is my capstone project: a full-stack portfolio and blog application.

It has two parts — a REST API built with Node.js and Express, and a single-page application built with React. The site is deployed online, and the source code has been submitted.

In the next few minutes I will explain how the system is designed, walk you through a live demo, and then talk about what I learned while building it.

2. What it is (1:00)

[Slide 2: two screenshots]

Let me start with what it actually looks like. On the left is the public side: a home page, a projects gallery, a blog list with article pages, and a contact form. On the right is the admin dashboard, which you can only reach after logging in. From there I can create, edit and delete both projects and blog posts.

So the product is three things in one: **a portfolio, a blog, and a content management dashboard**.

I broke the assignment down into three questions a real user would ask.

First, how does a visitor see my content? That needs public read endpoints.

Second, how do I manage that content? That needs write endpoints that only I can use, plus an admin interface.

Third, how do other people interact with me? That needs comments, and a contact form that actually stores messages in the database.

Those three questions map onto three layers of the system: **read, write, and permission**.

3. Tech stack (1:00)

[Slide 3: stack diagram]

On the front end I use React 19 with Vite, React Router 7 for routing, Axios for HTTP, the built-in Context API for global state, and hand-written CSS variables for styling — no UI framework.

On the back end: Node.js with Express 5, MongoDB Atlas as the database, Mongoose for modelling and validation, JWT for authentication, bcrypt for password hashing, helmet for security headers and dotenv for configuration.

One thing I want to stress: **I did not add tools just to have them**. My only global state is the logged-in user, so Context is enough — Redux would be overkill. And instead of importing a component library, I built a small design system on CSS variables, which stays lightweight and gives me light and dark themes for free.

4. Architecture (1:30)

[Slide 4: folder tree]

The back end follows the MVC pattern strictly.

`models` holds five Mongoose schemas — they define the data shape and the validation.

`controllers` holds the business logic, one file per resource.

`routes` is deliberately thin: it only maps URLs to controllers.

`middleware` contains the two pieces that matter most: `protect`, which verifies the token, and `error`, which formats every failure.

`utils` holds the error class and the token generator, `config/db.js` connects to the database, `app.js` assembles the application, and `server.js` does one thing: connect, then listen.

The two halves are completely separate projects. They talk only over HTTP under `/api`. The front end does not know what the database looks like, and the back end does not know what a page looks like. That means I can develop and deploy them independently.

5. Data models (2:00)

[Slide 5: ER diagram]

There are five models, and what matters most is how they relate.

`User` is the centre. `username` and `email` are both unique, the password has a minimum length, and it is declared with `select: false` — a normal query never returns the hash. Only the login query asks for it explicitly with `select('+password')`.

`Project` belongs to a user through an ObjectId that references `User`.

`BlogPost` has an `author` pointing at `User`.

`Comment` is the most interesting one: it points at **both** a user and a post — who said what, under which article — so it carries two references.

`Message` is the contact form result. It has no relation to users, because visitors do not need an account to write to me.

When reading, I restore those relationships with `populate`: the project list comes back with the author's username, the blog list too, and a single post comes back with all of its comments, each of them carrying the commenter's username.

[Slide 6: populate before/after]

This is the difference. The database stores IDs; the API returns objects the UI can render directly, so the front end does not need a second request.

6. Authentication and authorization (2:00)

[Slide 7: register/login flow]

Authentication has three steps.

At registration, the user submits a username, an email and a password. A `pre('save')` hook on the model hashes the password with bcrypt at cost factor 10, so the plain password never reaches the database.

At login, I fetch the user by email — this time **including** the password field — and compare with `bcrypt.compare`. If it matches, I sign a JWT containing only the user ID, valid for 30 days, using a secret from an environment variable.

After that, every protected request carries `Authorization: Bearer <token>`. The `protect` middleware verifies the signature, loads the user, and attaches it as `req.user`. A missing or invalid token gets a 401 immediately.

[Slide 8: 403 screenshot]

Authorization is a different question, and this is where a lot of people mix the two up. Authentication answers *who you are*. Authorization answers *what you are allowed to do*.

My rule is: **only the author can update or delete a post**. So in `updatePost` and `deletePost` I compare `post.author` with `req.user._id` and return 403 if they differ.

I tested this deliberately with two accounts: bob tries to edit alice's post, the API answers 403, and the post is unchanged.

7. API endpoints (1:30)

[Slide 9: endpoint table]

The API exposes 16 business endpoints plus a health check.

Three for users: register, login, and `/api/users/me`, which I added so the client can re-validate a stored token after a page refresh.

Five for projects: public list and detail, protected create, update and delete.

Five for posts: again two public, three protected — and update and delete also check authorship.

Two for comments: public read, authenticated write.

One for the contact form: public, so anyone can reach me.

Responses follow REST conventions: 201 with the created resource, a small success object on delete, and errors always `{ success: false, message }` with the correct status code.

8. Error handling and security (1:00)

[Slide 10: error middleware code]

Every controller does the same thing when something goes wrong: `next(err)`. All formatting happens in one central error middleware at the end of the chain. It translates the common cases into readable messages:

- a Mongoose validation failure becomes a 400 that names the offending field;
- a duplicate email or username — MongoDB error 11000 — becomes a 400 that says which field collided;
- an invalid ObjectId or a missing document becomes a 404;
- a malformed JSON body becomes a 400;
- an invalid or expired token becomes a 401;
- anything else becomes a 500, logged on the server but not leaked to the client.

On security: helmet sets safe response headers, CORS allows only the configured front-end origin, all secrets live in `.env` and never in the repository, request bodies are capped at 1 MB, and `trust proxy` is enabled so the app behaves correctly behind a reverse proxy such as Render.

9. Front-end architecture (1:30)

[Slide 11: routing and component map]

The client is a standard React single-page app with nine pages: home, projects, blog list, article detail, contact, login, register, admin, and a 404.

The public shell — header and footer — is extracted into a `Layout` component, and the pages render inside its `Outlet`.

My only global state is the session, kept in `AuthContext`: logging in stores the user and token in state and localStorage, logging out clears them. On start-up the provider calls `/api/users/me` once to check that the stored token is still valid, and logs the user out automatically if it is not. Without that check you end up in a fake "logged in" state where every write silently fails.

Every request goes through one Axios instance. A request interceptor attaches the `Authorization` header, and a response interceptor clears the local session on a 401 so the UI falls back to the logged-out state.

Protected pages are wrapped in `ProtectedRoute`: if there is no user it renders `<Navigate>` to the login page, remembers where the user wanted to go, and sends them back there after a successful login.

10. Design and interaction (1:00)

[Slide 12: design system]

For the visual layer I built a small design system on CSS variables: an indigo-to-magenta gradient as the primary colour, with cyan, amber and green as accents, over a soft coloured glow and a faint grid.

Light and dark themes are just two sets of variables, so the whole site switches at once. The choice is remembered in localStorage and defaults to the operating-system preference.

On interaction: loading states are skeletons instead of the word "Loading"; empty lists show a friendly empty state with a call to action; article pages have a reading-progress bar; destructive actions in the admin ask for confirmation; and on narrow screens the navigation becomes a single horizontally scrollable row.

All animation respects `prefers-reduced-motion`, so a user who turns motion off gets a completely static page that is still fully readable.

11. Live demo (3:00)

[Switch to the browser and follow the steps, narrating as you go]

Let me run through the whole flow from scratch.

Step one: register. On the home page I click Register, and fill in a username, an email and a password. After submitting, I am logged in automatically and taken to the dashboard — the client calls the login endpoint straight after registering, so the user does not type the password twice.

Step two: publish a project. In the Projects tab I fill in a title, a description and an image URL, then press Create. Notice the green confirmation at the top, and the new row appearing in the list on the right immediately.

Step three: check it on the public side. I open Projects, and the project is there with my username attached — that is `populate` at work.

Step four: write a post and comment. I publish a short article, open its detail page, and leave a comment. The comment appears straight away with my username and the date.

Step five: prove the permissions. I log out and go to `/admin` directly — I am redirected to the login page. I log in as another user and try to edit the article I just wrote: the API answers 403.

Step six: the contact form. I fill in a name, an email and a message, and submit. The success message appears, and the message now exists as a `Message` document in the database.

Finally, the theme toggle. I click the moon in the header; the whole palette switches, and after a refresh it is still dark.

12. What was hard (1:00)

[Slide 13]

Three things stayed with me.

First, **the difference between authentication and authorization**. At the beginning I assumed "logged in" meant "allowed to edit". Testing with a second account showed they are two separate decisions. Splitting "who you are" from "what you may do" actually made the controllers simpler to read.

Second, **deep links in a single-page app**. Refreshing `/blog/xxx` returns 404 because no such file exists on the server. The fix is a rewrite rule — `_redirects` on Netlify, `vercel.json` on Vercel — sending everything back to `index.html`.

Third, **cross-origin problems after deployment**. Locally the front end runs on 5173 and the API on 5000, so CORS has to allow the front-end origin. Once the front end moved to a Vercel domain, the back end's environment variable had to change with it. That is when environment variables stopped being optional for me.

13. Deployment and submission (0:45)

[Slide 14]

The API is deployed on Render: root directory `server`, start command `node server.js`, with the database URI, the JWT secret and the front-end URL as environment variables.

The client is deployed on Vercel: build command `npm run build`, output directory `dist`, and `VITE_API_URL` pointing at the deployed API.

The database is MongoDB Atlas, with Network Access opened for the host.

The submission is a zip containing `server`, `client` and a README. The README lists both live URLs, every endpoint, and the steps to run the project locally. As required, `node_modules`, `dist` and `.env` are excluded.

14. Closing (0:30)

To sum up: the project covers everything the brief asked for — an MVC REST API, five models with relationships, bcrypt and JWT, protected routes with authorisation, comments and a contact form, a component-based React client, React Router with a protected route, Context for global state, and separate deployment of both halves.

The biggest thing I take away is this: **the front end decides what people see, the back end decides what people can do, and the permission design decides whether the system can be trusted.**

That is my presentation. Thank you — I am happy to take questions.

PART 3 · Q&A preparation / 问答准备

Q1. Why MongoDB instead of a relational database?

中文： 我的数据结构是「文章属于作者、评论属于文章和用户」，本质上是几个一对多的关系，用文档数据库配合 `populate` 表达得很直接；而且 MongoDB Atlas 免去了自己维护数据库服务器的工作，部署到云端只要一个连接串。如果以后需要强事务或者复杂联表统计，我会考虑 PostgreSQL。

English: My data is a few one-to-many relationships — a post belongs to an author, a comment belongs to a post and a user. Documents plus `populate` express that directly, and Atlas means I do not have to run a database server myself: deployment is one connection string. If I needed strict multi-document transactions or heavy joins, I would move to PostgreSQL.

Q2. Why JWT instead of server-side sessions?

中文： 前后端是分开部署的，API 在 Render、前端在 Vercel，属于不同域名。JWT 把身份放在 token 里，API 不需要维护会话存储，也不需要共享 Cookie；横向扩容时天然是无状态的。代价是 token 一旦签发就无法立刻作废，所以我把有效期设成 30 天，并且前端在启动时用 `/users/me` 校验一次。

English: The two halves are deployed on different domains — Render for the API, Vercel for the client — so cookie-based sessions are awkward. A JWT carries the identity itself, the API stays stateless, and it scales horizontally without shared session storage. The trade-off is that a token cannot be revoked instantly, so I keep the lifetime at 30 days and validate it once at start-up through `/users/me`.

Q3. How are passwords stored? Is that safe?

中文： 数据库里只有 bcrypt 哈希，成本因子 10，从不存明文；`password` 字段声明了 `select: false`，普通查询根本拿不到它，只有登录时显式取出来做比较。所以即使有人读到了数据库，也拿不到可用的密码。当然生产环境还应该加上登录限流和 HTTPS，这些我在 README 里列成了后续改进。

English: Only a bcrypt hash with cost factor 10 is stored — never the plain password. The field is declared `select: false`, so ordinary queries cannot even return it; only the login query fetches it for comparison. Even with a database leak, the passwords are not directly usable. For production I would also add rate limiting on login and enforce HTTPS — I list both as future work in the README.

Q4. How do you stop one user from deleting another user's post?

中文： 两个层面。第一层是 `protect`：没有有效 token 一律 401，所以匿名用户连入口都没有。第二层在 controller 里：查出文章后比较 `post.author` 和 `req.user._id`，不一致返回 403。我特意用 bob 的账号去改 alice 的文章验证过，接口返回 403 且数据没变。

English: Two layers. First, `protect` rejects any request without a valid token, so anonymous users cannot even reach the handler. Second, the controller loads the post and compares `post.author` with `req.user._id`, returning 403 when they differ. I verified it by logging in as bob and trying to edit alice's post: 403, and the document was untouched.

Q5. What happens when the token expires?

中文： 后端 `jwt.verify` 会抛错，`protect` 捕获后返回 401；前端 Axios 的响应拦截器看到 401 就清掉 localStorage 里的登录信息并广播事件，界面立刻回到未登录状态，受保护的路由会把用户送去登录页。所以用户不会卡在一个「看起来登录了但什么都提交不了」的状态里。

English: On the server `jwt.verify` throws, `protect` catches it and answers 401. On the client, the Axios response interceptor sees the 401, clears the stored session and broadcasts an event, so the UI drops back to the logged-out state and the protected route redirects to the login page. The user never sits in a fake logged-in state.

Q6. Why Context API and not Redux?

中文： 因为全局状态只有一个：当前登录用户和他的 token。Context 加两个函数（login / logout）就够用了，不需要额外的库、action 类型和 reducer。如果以后要管更多跨页面状态，比如草稿、通知或者多语言，再引入状态管理库也来得及。

English: Because there is exactly one piece of global state: the current user and the token. Context with a `login` and a `logout` function covers it, with no extra dependency, action types or reducers. If the app grew drafts, notifications or i18n, I would introduce a state library then.

Q7. Is hiding `/admin` behind a login enough?

中文： 前端只是体验层，真正的安全在后端。就算有人绕过 React 直接调接口，所有写操作仍然要过 `protect`，文章的改删还要过作者校验。前端的 `ProtectedRoute` 只是避免未登录用户看到空白的后台界面。

English: The front end is only the experience layer; the real protection is on the server. Even if someone bypasses React and calls the API directly, every write still goes through `protect`, and post edits also go through the author check. `ProtectedRoute` only prevents logged-out visitors from seeing an empty dashboard.

Q8. What would you improve if you had another week?

中文： 四件事：一，给列表接口加分页和搜索，数据量大了才不会一次拉全量；二，加 refresh token 和登出黑名单，让会话可以真正作废；三，支持图片上传而不是只填 URL；四，给后台加一个「消息」标签页，直接在界面里读联系表单的留言。

English: Four things. Pagination and search on the list endpoints, so they do not return everything at once. A refresh-token flow with revocation, so sessions can actually be invalidated. Real image uploads instead of pasting a URL. And a "Messages" tab in the admin so I can read contact-form messages inside the app.

Q9. How would this scale?

中文： 应用层本身是无状态的，所以横向加实例就行；数据库用 Atlas 的副本集，读写分离由官方托管。真正的瓶颈会在「每次列表都全量返回」和「文章详情的两次查询」，所以我会先做分页和索引，再看要不要加缓存。

English: The app tier is stateless, so scaling out just means more instances. Atlas already handles replication. The real bottlenecks would be unbounded list queries and the extra query on a post detail page, so I would add pagination and indexes first, then consider caching.

Q10. What did you personally find hardest?

中文： 把「看起来能跑」变成「验证过能跑」。写完之后我写了一套 35 项的接口检查，把注册、登录、权限、404、重复键这些边界都跑了一遍，才发现几个原来没注意的问题——比如 token 过期后的状态处理。测试比写功能更花时间，但也更值得。

English: Turning "it seems to work" into "I verified it works". After building it I wrote a 35-check script that exercises registration, login, permissions, 404s and duplicate keys — and that is how I found the edge cases I had missed, such as what happens after a token expires. Testing took longer than writing the features, and it was worth it.

PART 4 · 附录：接口速查 / API quick reference

Method	Endpoint	Access	说明 / Notes
POST	`/api/users/register`	Public	注册，密码哈希后入库 / creates a user with a hashed password
POST	`/api/users/login`	Public	返回 JWT / returns a JWT
GET	`/api/users/me`	Bearer	校验本地 token / validates a stored token
GET	`/api/projects`	Public	列表，带作者名 / list with

			author username
GET	`/api/projects/:id`	Public	单个项目 / one project
POST	`/api/projects`	Bearer	新建，绑定当前用户 / creates a project for `req.user`
PUT	`/api/projects/:id`	Bearer	更新 / partial update
DELETE	`/api/projects/:id`	Bearer	删除 / delete
GET	`/api/blog`	Public	文章列表，带作者名 / list with author
GET	`/api/blog/:id`	Public	文章 + 全部评论 / post with comments
POST	`/api/blog`	Bearer	新建文章 / creates a post
PUT	`/api/blog/:id`	Bearer + author	仅作者可改 / author only, else 403
DELETE	`/api/blog/:id`	Bearer + author	仅作者可删，并删评论 / author only
GET	`/api/blog/:postId/comments`	Public	某篇的评论 / comments of a post
POST	`/api/blog/:postId/comments`	Bearer	发表评论 / add a comment
POST	`/api/contact`	Public	联系表单入库 / stores a Message
GET	`/api/health`	Public	健康检查 / health check

状态码 / Status codes

Code	场景 / When
200	读取或更新成功 / read or update succeeded
201	创建成功 / resource created
400	参数缺失、校验失败、重复键、非法 JSON / validation or bad input
401	没有 token 或 token 无效 / missing or invalid token
403	已登录但不是作者 / authenticated but not the author
404	资源不存在 / resource not found
500	未预期的服务端错误 / unexpected server error

PART 5 · 术语对照 / Glossary

中文	English	一句话解释
全栈	full-stack	同时负责前端和后端
单体 / 前后端分离	decoupled SPA + API	两个独立部署的工程，只通过 HTTP 通信
路由	route	URL 到处理函数的映射
中间件	middleware	在请求到达处理函数前后执行的一层函数
校验	validation	入库前检查数据是否合法
哈希	hash	单向加密，只能比对不能还原
签发令牌	sign a token	用密钥给 token 签名，防止被伪造
鉴权	authorization	判断「你能不能做这件事」
关联查询	populate	把存着的 ID 展开成完整对象
单页应用	SPA	只加载一次 HTML，之后由 JS 切换页面
受保护路由	protected route	未登录时重定向的路由
全局状态	global state	多个页面共享的数据
骨架屏	skeleton	加载时占位的灰块
深链接	deep link	直接访问某个内页地址
环境变量	environment variable	不写进代码的配置，比如密钥
部署	deploy	把代码放到线上服务器运行